

AWS BUILDER'S PERSPECTIVE

Shifting Security Left

Building Secure CI/CD for Multi-Cloud

An AWS Builder's Perspective on OpenTUF, Terraform & Modern Supply Chain Security



AWS Native



OpenTUF



Terraform



CI/CD

VECTOR ILLUSTRATION
BIGDATA PROCESSING

AGENDA

What We'll Cover Today

01

The Security Left Shift Imperative

Why traditional late-stage security fails in cloud-native environments

02

AWS CI/CD Security Architecture

CodePipeline, CodeBuild & CodeDeploy with security controls

03

Terraform Security in CI/CD

State management, OIDC auth & security scanning tools

04

OpenTUF & Supply Chain Security

The Update Framework, TUF roles & key hierarchy

05

Container Security & Image

Signing Cosign & keyless signing with OIDC

06

Multi-Cloud Security Patterns

Zero trust, unified identity & consistent policies

07

Implementation Roadmap & Best Practices

Actionable phases from quick wins to advanced security controls

The Security Left Shift Imperative

⚠️ Why Traditional Security Fails

Traditional security operates as a **gatekeeper**, creating bottlenecks and missing vulnerabilities that are exponentially more expensive to fix late in the lifecycle.

100x

Cost to fix in production vs development

2021

SolarWinds attack accelerated adoption

🛡️ The DevSecOps Paradigm

DevSecOps embeds security as a **continuous practice** across the entire development lifecycle, automating security checks and providing immediate feedback to developers.

Shift Left

Continuous

Collaborative

Three-Layer Defense Model

1 Prevent

IDE plugins, pre-commit hooks, secrets scanning. Catch issues before commit.

2 Detect

CI pipeline scanning—SAST, DAST, dependency checks, IaC validation.

3 Protect

Runtime security, admission controllers, network policies, monitoring.

💡 Key Insight

Organizations implementing shift-left security reduce production vulnerabilities by **70%** and cut remediation time from weeks to hours.

AWS CI/CD Security Architecture

Pipeline Security Stack

Source Stage

CodePipeline retrieves code from GitHub/CodeCommit with webhook triggers

Signed
Commits

Branch
Protection

Build Stage

CodeBuild compiles, tests, and packages in isolated VPC environments

VPC Isolation

Security
Scanning

Deploy Stage

CodeDeploy automates deployments to EC2, ECS, Lambda with rollback

Blue/Green

Approval Gates

IAM Best Practices



Least Privilege

Each service has dedicated IAM role with minimal required permissions—no wildcards



Separate Roles

CodePipeline, CodeBuild, CodeDeploy each have distinct service roles



Cross-Account

Use sts:AssumeRole for deploying to production accounts from CI/CD account



Artifact Security

- ✓ S3 buckets with SSE-KMS encryption
- ✓ Block public access
- ✓ Lifecycle policies for cleanup

Critical: Your CI/CD pipeline has production access—secure it with the same rigor as production itself. Enable CloudTrail logging for all pipeline actions.

Terraform Security: State & Authentication

Secure State Management

```
terraform {
  backend "s3" {
    bucket     = "tf-state-prod"
    key        = "infra.tfstate"
    region     = "us-east-1"
    encrypt    = true
    kms_key_id = "arn:aws:kms:...:dynamodb_table = "tf-lock"
```

Encryption at Rest

State files contain secrets—always enable SSE-KMS encryption with customer-managed keys

State Locking

DynamoDB prevents concurrent modifications that corrupt state—mandatory for team workflows

State Sensitivity

Terraform state contains all resource data including sensitive values—treat as confidential

OIDC Authentication

Eliminate long-lived AWS credentials entirely. Use GitHub Actions for short-lived, automatically rotated tokens.

```
# GitHub Actions OIDC
- uses: aws-actions/configure-aws-credentials@v4
  with:
    role-to-assume: arn:aws:iam::...:role/tf-ci
    aws-region: us-east-1
```

No Static Credentials

Tokens expire after job completion—no secrets to leak or rotate

Branch Protection

IAM trust policy restricts role assumption to specific repos/branches

Secrets Management

- ✓ AWS Secrets Manager for dynamic secrets
- ✓ Parameter Store for configuration
- ✓ Never commit .tfvars with secrets

Best Practice: Use Terraform workspaces for environment isolation (dev/staging/prod) with separate state files and IAM roles per workspace.

Terraform Security Scanning in CI/CD



Fast static analysis for Terraform security.
Catches misconfigurations before they reach production.

```
tfsec . --severity HIGH
# Output: CRITICAL
# AWS017: Resource defines fully
# open ingress security group rule
```

What It Catches

- Open security groups (0.0.0.0/0)
- Unencrypted EBS/RDS/S3
- Missing logging/monitoring

CI/CD Integration

Run on every PR. Fail builds on HIGH/CRITICAL findings.



Recommended Pipeline Order

TFLint → **tfsec** → **Checkov (plan)** . Fail fast on code issues, then deep scan the actual changes.



Comprehensive policy-as-code scanner with 1000+ built-in checks for compliance (CIS, NIST, SOC2).

```
checkov -d . --framework terraform
# Scan Terraform plan
terraform plan -out=plan.tfplan
checkov -f plan.json \
  --framework terraform_plan
```

Compliance Coverage

- CIS AWS Foundations Benchmark
- NIST 800-53 controls
- SOC 2, GDPR, HIPAA

Custom Policies

Write YAML/Python policies for organizational requirements.



Terraform linter for code quality, best practices, and provider-specific rules.

```
tflint --init
tflint
# Catches: deprecated args,
# unused variables, naming
# convention violations
```

Code Quality

- Unused variables/resources
- Deprecated arguments
- Provider-specific rules

Team Consistency

Enforce naming conventions and module standards across teams.

OpenTUF: The Update Framework Explained

What is TUF?

The Update Framework (TUF) is a **framework** that protects against compromised repositories, man-in-the-middle attacks, and key theft.

Key Capabilities

- ▶ **Survivable Key Compromise:** Role separation prevents single key loss from being fatal
- ▶ **Freshness Guarantees:** Timestamp metadata prevents replay attacks with old versions
- ▶ **Trust Thresholds:** Require multiple signatures for critical operations

Docker Content Trust used TUF via Notary for image signing. As of 2025, Docker is deprecating DCT in favor of modern alternatives like Sigstore.

TUF Key Roles Hierarchy

R Root Role

Top-level trust anchor. Certifies other role keys. Kept offline, highest security.

T Targets Role

Signs actual software artifacts and their hashes. Maps filenames to trusted hashes.

S Snapshot Role

Signs metadata about all targets metadata versions. Prevents mix-and-match attacks.

TS Timestamp Role

Short-lived signatures ensuring metadata freshness. Rotated frequently (daily/hourly).

Industry Adoption: TUF powers **PyPI** (Python), **RubyGems**, **Containerd**, and **Notary**. It's the foundation of modern software supply chain security.

Sigstore & Cosign: Keyless Container Signing

Why Sigstore?

Sigstore eliminates the complexity of key management. Instead of managing long-lived signing keys, it uses  with OIDC identity—your GitHub, Google, or Microsoft account becomes your signing key.

Sigstack Components

-  **Cosign**
CLI tool for signing and verifying containers
-  **Fulcio**
Free, automated certificate authority
-  **Rekor**
Transparency log for all signatures

No Key Management: Signatures are tied to your identity, not a key file. No rotation, no leaks, no hassle.



Enterprise Option: AWS KMS Integration

For organizations requiring hardware-backed keys: `cosign sign --key aws:kms:///alias/my-key`

Keyless Signing in CI/CD

```
# GitHub Actions - Sign with OIDC
- uses: sigstore/cosign-installer@v3
- name: Sign image
  run: |
    cosign sign --yes \
      $REGISTRY/myapp:${{ github.sha }}
```

```
# Verify signature before deploy
cosign verify \
  --certificate-identity= \
    "https://github.com/myorg/..." \
  --certificate-oidc-issuer= \
    "https://token.actions..." \
  $IMAGE
```

Kyverno Policy Enforcement

```
verifyImages:
- imageReferences:
  - "ghcr.io/myorg/*"
  attestors:
  - keyless:
    subject: "https://github.com/..."
    issuer: "https://token..."
```


SLSA & Supply Chain Attestations

SLSA Framework

Supply Chain Levels for Software Artifacts (SLSA) is a [redacted] providing checklists and standards to prevent tampering and secure software supply chains.

Level

1

Build process must generate provenance showing how artifact was built

Basic

Level 2

Hosted build platform generates and signs provenance using automated CI/CD

Signed

Level 3

Isolated builds in ephemeral environments with protection against insider threats

Hardened

in-toto Attestations

in-toto provides a [redacted] through cryptographically signed attestations at each CI/CD step.

Attestation Chain

1

Source attestation (signed commit)

2

Build attestation (what was built, by whom)

3

Scan attestation (security check results)

4

Deploy attestation (deployment approval)

Verification: Admission controllers verify the complete attestation chain before allowing deployments.

Implementation: Use **SLSA GitHub Generator** for automated provenance generation and **slsa-verifier** for verification. Start with SLSA L2 and work toward L3.

Multi-Cloud Security Architecture Patterns

Zero Trust Architecture

Assume  by default. Every access request must be verified regardless of location.

Identity-First Security

Centralized identity across AWS, Azure, GCP using OIDC and workload identity federation

Network Segmentation

Hub-and-spoke model with shared security services. VPC endpoints, private connectivity.

Least Privilege

Granular IAM policies, just-in-time access, continuous validation of permissions

Unified Security Controls

Policy as Code

OPA/Gatekeeper enforce consistent policies across all clouds

```
deny[msg] { input.request.kind.kind == "Pod" ... }
```

Centralized Secrets

HashiCorp Vault or cloud-native solutions with cross-cloud replication

Unified Observability

Centralized logging, monitoring, and audit trails across all environments

Hub-and-Spoke

Shared security services in hub

Workload Identity

Federation across cloud providers

IaC Scanning

Scan all cloud configs pre-deploy

Implementation Roadmap & Best Practices

Phased Implementation

Phase 1: Quick Wins (Weeks

1-2)

- ✓ Enable OIDC authentication for all CI/CD pipelines
- ✓ Encrypt Terraform state with KMS, enable DynamoDB locking
- ✓ Add trivy scanning to all Terraform repos (fail on CRITICAL)
- ✓ Enable AWS Config and Security Hub for compliance monitoring

High
Impact

Phase 2: Pipeline Hardening (Weeks

3-6)

- ✓ Implement container signing with Cosign (keyless)
- ✓ Add Trivy vulnerability scanning for all container images
- ✓ Generate SLSA L2 provenance with GitHub Actions
- ✓ Deploy Kyverno for admission control (signed images only)

Medium
Effort

Phase 3: Advanced (Months 2-

3)

- ✓ Achieve SLSA L3 with hardened builds and ephemeral environments
- ✓ Implement in-toto attestations for full supply chain verification
- ✓ Deploy OPA/Gatekeeper for cross-cloud policy enforcement

Long-term

Security Checklist

- ✓ No secrets in Git (use Secrets Manager)
- ✓ Least-privilege IAM (no wildcards)
- ✓ Encrypted artifacts at rest (KMS)
- ✓ Signed container images (Cosign)
- ✓ Scanned IaC (tfsec/Checkov)
- ✓ VPC-isolated builds
- ✓ Audit logging (CloudTrail)
- ✓ Automated rollback on failure

⚠ Common Pitfalls

- Storing AWS credentials in GitHub secrets
- Running builds without VPC isolation
- Skipping security scans for "quick fixes"

KEY TAKEAWAYS

Security is Not a Gate— It's a Continuous Practice



Shift Left

Catch vulnerabilities in development, not production. Use IDE plugins, pre-commit hooks, and CI scanning.



Use OIDC

Eliminate long-lived credentials entirely. Short-lived tokens with automatic rotation are the gold standard.



Sign Everything

Code commits, container images, infrastructure changes. Verification at every stage creates a chain of trust.



Adopt SLSA Incrementally

Start with provenance (L1), add signing (L2), then hardened builds (L3). Don't try to achieve everything at once.



Remember

Your CI/CD pipeline is production—secure it with the same rigor. A compromised pipeline means compromised everything.